

## Autenticación de Usuarios con JSON Web Tokens y Prisma

### Resumen

La autenticación de usuarios es un componente fundamental en el desarrollo de aplicaciones web modernas. Implementar un sistema robusto que permita a los usuarios registrarse, iniciar sesión y acceder a rutas protegidas es esencial para garantizar la seguridad de nuestras aplicaciones. En este contenido, exploraremos cómo crear un endpoint de login funcional utilizando Node.js, Prisma y JSON Web Tokens.

### ¿Cómo implementar un endpoint de login seguro?

Cuando desarrollamos una API, necesitamos crear un sistema de autenticación que permita a los usuarios acceder a recursos protegidos. Para lograr esto, vamos a implementar un endpoint de login que verificará las credenciales del usuario y generará un token de autenticación.

Comencemos creando nuestro endpoint en el archivo principal:

```
app.post('/login', async (req, res) => {  
  // Extraer email y password del cuerpo de la solicitud  
  const { email, password } = req.body;  
  
  // Buscar el usuario en la base de datos  
  const user = await prisma.user.findUnique({  
    where: { email }  
  });  
  
  // Validar si el usuario existe  
  if (!user) {  
    return res.status(400).json({ error: "Invalid email or password" });  
  }  
  
  // Verificar si la contraseña coincide  
  const validPassword = await bcrypt.compare(password, user.password);  
  
  // Validar si la contraseña es correcta  
  if (!validPassword) {  
    return res.status(400).json({ error: "Invalid email or password" });  
  }  
})
```

Este código realiza varias operaciones importantes:

1. **Extrae las credenciales** del cuerpo de la solicitud
2. **Busca al usuario** en la base de datos usando Prisma
3. **Verifica la contraseña** utilizando bcrypt
4. **Genera un token JWT** con información relevante del usuario
5. **Devuelve el token** al cliente para su uso posterior

### ¿Por qué es importante la seguridad en los mensajes de error?

Un aspecto crucial de la seguridad en la autenticación es **no revelar información específica en los mensajes de error**. Observa que en ambas validaciones

(usuario inexistente y contraseña incorrecta) devolvemos el mismo mensaje: "Invalid email or password".

Esto es una práctica de seguridad fundamental porque:

- Evita que atacantes determinen si un correo electrónico específico está registrado
- No proporciona pistas sobre cuál de las credenciales es incorrecta
- Dificulta los ataques de fuerza bruta o de enumeración de usuarios

Si mostráramos mensajes específicos como "El correo no existe" o "La contraseña es incorrecta", estaríamos dando información valiosa a posibles atacantes.

¿Cómo funciona la verificación de contraseñas con bcrypt?

La verificación de contraseñas es un proceso crítico en cualquier sistema de autenticación. En nuestro código, utilizamos bcrypt para comparar la contraseña proporcionada con la almacenada en la base de datos:

```
const validPassword = await bcrypt.compare(password, user.password);
```

Esta línea hace todo el trabajo pesado:

- Toma la contraseña en texto plano del request
- La compara con la versión hasheada almacenada en la base de datos
- Devuelve true si coinciden, false si no

**bcrypt** es una biblioteca especializada que implementa el algoritmo Blowfish para el hashing de contraseñas, diseñado específicamente para ser lento y resistente a ataques de fuerza bruta.

¿Qué son los JSON Web Tokens y cómo implementarlos?

Los JSON Web Tokens (JWT) son un estándar abierto que define una forma compacta y autónoma de transmitir información de forma segura entre partes como un objeto JSON. Esta información puede ser verificada porque está firmada digitalmente.

En nuestro código, generamos un token JWT con la siguiente estructura:

```
const token = jwt.sign(  
  { id: user.id, role: user.role },  
  process.env.JWT_SECRET,  
  { expiresIn: '4h' }  
);
```

Este token contiene:

1. **Payload:** Información del usuario (ID y rol)
2. **Firma:** Creada con una clave secreta almacenada en variables de entorno
3. **Tiempo de expiración:** Configurado para 4 horas

El token generado será utilizado por el cliente para acceder a rutas protegidas. Cada vez que el cliente realice una solicitud a una ruta protegida, deberá incluir este token en los encabezados de la solicitud.

¿Cómo proteger rutas con middleware de autenticación?

Para proteger ciertas rutas de nuestra API, necesitamos implementar un middleware que verifique la validez del token JWT. Este middleware se ejecutará antes de acceder a las rutas protegidas:

```
const authenticateToken = (req, res, next) => {
  const token = req.headers['authorization'];

  if (!token) {
    return res.status(401).json({ error: "Access denied" });
  }

  try {
    const verified = jwt.verify(token, process.env.JWT_SECRET);
    req.user = verified;
    next();
  } catch (error) {
    res.status(400).json({ error: "Invalid token" });
  }
};

// Ejemplo de ruta protegida
app.get('/protected', authenticateToken, (req, res) => {
  res.json({ message: "Esta ruta es protegida" });
});
```

Este middleware:

1. **Extrae el token** de los encabezados de la solicitud
2. **Verifica su validez** utilizando la clave secreta
3. **Adjunta la información del usuario** al objeto de solicitud para su uso posterior
4. **Permite continuar** con la solicitud si el token es válido

¿Cómo probar nuestra API de autenticación?

Para probar nuestra implementación de autenticación, podemos utilizar herramientas como Postman. El flujo de prueba sería:

1. **Registrar un usuario** enviando una solicitud POST a /register

2. **Iniciar sesión** enviando una solicitud POST a /login con las credenciales
3. **Copiar el token** recibido en la respuesta
4. **Acceder a una ruta protegida** enviando una solicitud GET a /protected incluyendo el token en los encabezados

Si todo funciona correctamente, deberíamos poder acceder a la ruta protegida y recibir la respuesta esperada.

Es importante recordar que en un entorno de producción, el frontend sería responsable de almacenar el token (generalmente en localStorage o cookies) y enviarlo en cada solicitud a rutas protegidas.

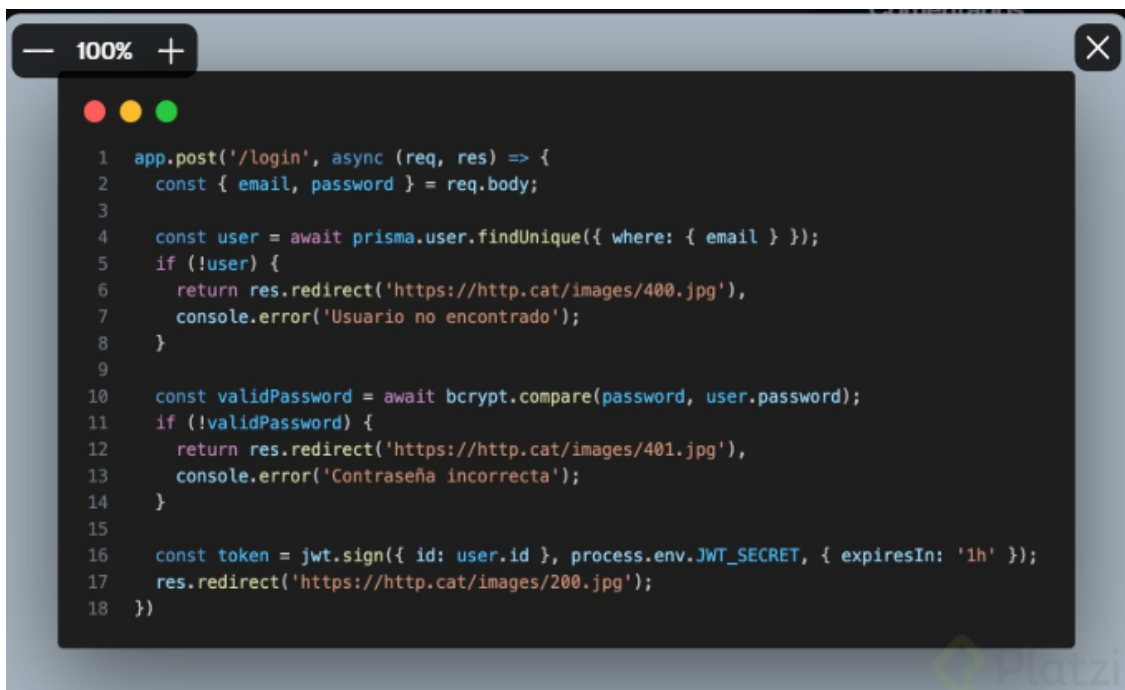
La implementación de un sistema de autenticación robusto es fundamental para proteger los recursos de nuestra aplicación. Siguiendo las prácticas recomendadas y utilizando bibliotecas probadas como bcrypt y jsonwebtoken, podemos crear un sistema seguro y confiable para nuestros usuarios.

¿Has implementado sistemas de autenticación en tus proyectos? ¿Qué desafíos has enfrentado? Comparte tus experiencias en los comentarios y sigamos aprendiendo juntos.

Yo en esta parte, remplace los mensaje de errores, por gatos.

Aparte de ser entretenida de mirar, ayuda a entender de una mejor manera, por que el error 400 es el que es.

Al menos para hacer más divertido el aprendizaje.

A screenshot of a code editor window with a dark theme. The window has a title bar with a close button (X) and a zoom level of 100%. The code is written in JavaScript and shows a login function. It uses Prisma to find a user by email and bcrypt to compare passwords. If a user is not found, it redirects to a cat image (400.jpg) and logs an error. If the password is incorrect, it redirects to another cat image (401.jpg) and logs an error. If the login is successful, it generates a JWT token and redirects to a third cat image (200.jpg).

```
1 app.post('/login', async (req, res) => {
2   const { email, password } = req.body;
3
4   const user = await prisma.user.findUnique({ where: { email } });
5   if (!user) {
6     return res.redirect('https://http.cat/images/400.jpg'),
7     console.error('Usuario no encontrado');
8   }
9
10  const validPassword = await bcrypt.compare(password, user.password);
11  if (!validPassword) {
12    return res.redirect('https://http.cat/images/401.jpg'),
13    console.error('Contraseña incorrecta');
14  }
15
16  const token = jwt.sign({ id: user.id }, process.env.JWT_SECRET, { expiresIn: '1h' });
17  res.redirect('https://http.cat/images/200.jpg');
18 })
```